AI DEVOPS CONFIGURATION REVIEWER: A HYBRID STATIC AND AI-POWERED SECURITY ANALYSIS PLATFORM

A PROJECT REPORT

Submitted by

DHVANI PATEL [RA2512005010044]

Under the Guidance of

Dr. SANTHIYA P

(Assistant Professor, Department of Computing Technologies)

in partial fulfilment of the requirements for the course

21CSE746P - AI INTEGRATED FULL STACK APPLICATION DEVELOPMENT WITH DEVOPS

MASTER OF TECHNOLOGY

in

COMPUTER SCIENCE ENGINEERING



DEPARTMENT OF COMPUTING TECHNOLOGIES

COLLEGE OF ENGINEERING AND TECHNOLOGY

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

KATTANKULATHUR - 603 203

NOVEMBER 2026

Department of Computing Technologies
SRM Institute of Science & Technology

Own Work Declaration Form

This sheet must be filled in, signed, dated with a student registration number, and included with all assignments submitted for evaluation.

Field	Details
Degree / Course	M.Tech / 21CSE746P - AI Integrated Full Stack Application Development with DevOps
Student Name	DHVANI PATEL
Registration Number	RA2512005010044
Title of Work	AI DevOps Configuration Reviewer: A Hybrid Static and AI-Powered Security Analysis Platform

I hereby certify that this assessment complies with the University rules and regulations relating to academic misconduct and plagiarism. I confirm that all work contained in this report is my own except where indicated through citation and acknowledgement, and that I have:

- Clearly referenced and listed all sources as appropriate.
- Referenced and put in inverted commas all quoted text from books, documentation, or other sources.
- Given the sources of all pictures, diagrams, data, and code that are not my own.
- Not made use of the report or essay of any other student either past or present.
- Acknowledged any help received from others.
- Complied with any other plagiarism criteria specified by the course handbook or University website.

DECLARATION: I am aware of and understand the University policy on academic misconduct and plagiarism. I certify that this assessment is my own work, except where indicated by referencing, and that I have followed good academic practices.

Student Signature	Student Name	Registration Number	Date
	DHVANI PATEL	RA2512005010044	

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR - 603 203

BONAFIDE CERTIFICATE

Certified that 21CSE746P - AI Integrated Full Stack Application Development with DevOps – Project report titled "AI DevOps Configuration Reviewer: A Hybrid Static and AI-Powered Security Analysis Platform" is the bonafide work of DHVANI PATEL [RA2512005010044] who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE	**SIGNATURE**
Dr. SANTHIYA P	**Dr. G. NIRANJANA**
ASSISTANT PROFESSOR	PROFESSOR & HOD
DEPARTMENT OF COMPUTING TECHNOLOGIES	DEPARTMENT OF COMPUTING TECHNOLOGIES

ACKNOWLEDGEMENT

I express my humble gratitude to **Dr. C. Muthamizhchelvan**, Vice-Chancellor, SRM Institute of Science and Technology, for the facilities extended for the project work and his continued support.

I extend my sincere thanks to **Dr. Leenus Jesu Martin M**, Dean-CET, SRM Institute of Science and Technology, for his invaluable support.

I wish to thank **Dr. Revathi Venkataraman**, Professor and Chairperson, School of Computing, SRM Institute of Science and Technology, for her support throughout the project work.

I encompass my sincere thanks to **Dr. M. Pushpalatha**, Professor and Associate Chairperson - CS, School of Computing, and **Dr. C. Lakshmi**, Professor and Associate Chairperson - AI, School of Computing, SRM Institute of Science and Technology, for their invaluable support.

I am incredibly grateful to our Head of the Department, **Dr. G. Niranjana**, Professor, Department of Computing Technologies, SRM Institute of Science and Technology, for her suggestions and encouragement at all the stages of the project work.

My inexpressible respect and thanks to my subject faculty and guide, **Dr. Santhiya P**, Assistant Professor, Department of Computing Technologies, SRM Institute of Science and Technology, for providing me with an opportunity to pursue my project under her mentorship. She provided me with the freedom and support to explore the research topics of interest, and her guidance at every review shaped both the system and this report.

I also wish to thank the members of the project review panel, Department of Computing Technologies, SRM Institute of Science and Technology, for their valuable feedback and insightful review of my work.

I sincerely thank all the staff members of the Department of Computing Technologies, School of Computing, SRM Institute of Science and Technology, for their help during my project. Finally, I would like to thank my parents, family members, and friends for their unconditional love, constant support and encouragement.

DHVANI PATEL

ABSTRACT

Modern software development relies heavily on Infrastructure-as-Code: Dockerfiles, Docker Compose manifests, Kubernetes configurations, GitHub Actions workflows, Terraform scripts and Helm charts. These files are version-controlled alongside application code and, if misconfigured, can expose production systems to privilege escalation, secrets leakage, open network access and supply-chain attacks. Existing scanners are either purely deterministic — fast but blind to contextual risk — or large-language-model wrappers that hallucinate findings and produce no structured output suitable for automated remediation pipelines.

This project builds AI DevOps Configuration Reviewer, a full-stack web application that submits any supported configuration file and receives structured security findings within seconds. A hybrid analysis pipeline first runs a deterministic static analyser that applies 48 hand-authored rules across six configuration types (Dockerfile, Docker Compose, GitHub Actions, Kubernetes, Terraform and Helm), then dispatches a background task that calls an OpenAI language model with a strict Pydantic JSON schema to produce AI findings with fix suggestions for every issue it identifies. A deterministic scoring engine — never the AI — computes a risk score in [0, 100] by summing severity-weighted finding counts, so the score is always reproducible and explainable. Users can chat with an AI expert about any individual finding, request a plain-English score explanation, view a trend chart of how their configuration's risk score has changed over time, upload and analyse multiple files in one bulk operation, and share read-only public report links.

The backend is an async FastAPI application backed by PostgreSQL 16 with SQLAlchemy 2 and six Alembic migrations. The frontend is a Next.js 16 App Router application written in strict TypeScript with Tailwind CSS v4. A provider-abstraction ABC makes it possible to swap OpenAI for a local Ollama model by changing one environment variable. The system is containerised with Docker, monitored with Prometheus and Grafana, and delivered through a three-job GitHub Actions CI pipeline that runs 112 backend tests with pytest and a TypeScript build on every push.

TABLE OF CONTENTS

ABSTRACT **v**

TABLE OF CONTENTS **vii**

LIST OF FIGURES **ix**

LIST OF TABLES **x**

ABBREVIATIONS **xi**

1 INTRODUCTION **1**

1.1 Introduction 1

1.2 Problem Statement 1

1.3 Motivation 2

1.4 Objectives 3

1.5 Course Outcome and SDG Alignment 3

1.6 Scope of the Report 4

1.7 Contributions 5

2 LITERATURE SURVEY **6**

2.1 Overview 6

2.2 DevSecOps and Shift-Left Security 6

2.3 Static Analysis Tools for Configuration Files 6

2.4 AI-Powered Code and Configuration Analysis 7

2.5 Container and Infrastructure Security Standards 8

2.6 Research Gaps and Project Objectives 8

3 PROPOSED METHODOLOGY **10**

3.1 Methodology Overview 10

3.2 System Architecture 10

3.3 System Components 11

3.4 Analysis Request Lifecycle 12

3.5 Static Analyser Engine 12

3.6 AI Analysis Pipeline 13

3.7 Risk Scoring Model 14

3.8 Mathematical Model Summary 14

4 FULL-STACK IMPLEMENTATION AND DEVOPS **15**

4.1 Design Principles 15

4.2 Backend REST API 16
4.3 Frontend Implementation 17
4.4 Containerisation 18
4.5 Continuous Integration 18
4.6 Observability and Monitoring 19
4.7 Testing Methodology 20
5 EXPERIMENTAL SETUP 21
5.1 Environment 21
5.2 Configuration Types and Rule Coverage 21
5.3 Test Suite 22
5.4 Evaluation Protocol 22
6 RESULTS AND DISCUSSION 23
6.1 Static Analysis Coverage 23
6.2 AI Finding Quality 24
6.3 Risk Score Validation 24
6.4 Feature Validation 25
6.5 Performance Metrics 25
6.6 Discussion and Limitations 26
7 CONCLUSION AND FUTURE ENHANCEMENTS 27
7.1 Conclusion 27
7.2 Future Enhancements 28
7.3 Final Remarks 29
REFERENCES 30
APPENDIX A — BACKEND MODULE STRUCTURE 33
APPENDIX B — REPOSITORY LAYOUT 34
APPENDIX C — APPLICATION SCREENSHOTS 35
APPENDIX D — SELECTED SOURCE CODE 38

LIST OF FIGURES

3.1	System architecture: Next.js frontend, FastAPI backend, PostgreSQL and AI provider	10
3.2	Analysis request lifecycle from HTTP submission to completed findings	12
3.3	Static analyser class hierarchy and rule execution flow	13
3.4	AI provider abstraction: OpenAI and Ollama implementations	14
4.1	CI/CD pipeline from git push to automated tests and Docker build	18
4.2	Prometheus and Grafana observability stack	19
6.1	Risk score distribution across 50 test configurations	23
6.2	Finding severity breakdown by configuration type	24
6.3	Score trend chart for a sample Kubernetes configuration over six analyses	25
C.1	Analysis results page with finding cards and risk score	35
C.2	Bulk upload page showing parallel analysis progress	36
C.3	Per-finding remediation chat panel	36
C.4	Public shareable report page (no login required)	37
C.5	Grafana dashboard with Prometheus metrics	38

LIST OF TABLES

1.1	Course outcome alignment for 21CSE746P	3
3.1	System components and responsibilities	11
3.2	Static analyser rules by configuration type	13
4.1	REST API endpoints of the backend	16
4.2	Frontend routes and their purpose	17
5.1	Test configuration files used for evaluation	22
5.2	Backend test coverage by module	22
6.1	Static rule hit rates on the test dataset	23
6.2	AI finding quality metrics (precision, recall, hallucination rate)	24
6.3	Risk score reproducibility across identical inputs	25
6.4	Identified limitations and planned corrections	26
A.1	Backend Python modules and their responsibilities	33
B.1	Repository layout	34

ABBREVIATIONS

Abbreviation	Expansion
ABC	Abstract Base Class
AI	Artificial Intelligence
API	Application Programming Interface
CD	Continuous Delivery / Deployment
CI	Continuous Integration
CO	Course Outcome
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSP	Cloud Service Provider
DB	Database
HTTP	Hyper Text Transfer Protocol
IaC	Infrastructure-as-Code
JSON	JavaScript Notation Object
JWT	JSON Web Token
LLM	Large Language Model
ORM	Object-Relational Mapper
OWASP	Open Web Application Security Project
REST	Representational State Transfer
SARIF	Static Analysis Results Interchange Format
SDG	Sustainable Development Goal
SQL	Structured Query Language
TLS	Transport Layer Security
UI	User Interface
UUID	Universally Unique Identifier
YAML	YAML Ain't Markup Language

CHAPTER 1
INTRODUCTION

1.1 Introduction

Modern cloud-native software is assembled from declarative configuration files rather than hand-configured servers. A developer who builds a containerised application writes a Dockerfile to describe the runtime image, a Docker Compose file to assemble multi-service stacks, a Kubernetes manifest to deploy those stacks to a cluster, a Terraform script to provision the underlying cloud infrastructure, a Helm chart to package the Kubernetes resources for distribution, and one or more GitHub Actions workflows to automate build, test and deploy operations. Each of these files travels in the same version-control repository as the application code, but unlike application code it is rarely subjected to the same depth of automated security review.

Misconfigurations in these files are a leading cause of cloud security incidents. A container run as the root user, a Kubernetes pod scheduled with host networking, a Terraform resource with an open security-group rule, or a GitHub Actions workflow that exposes GITHUB_TOKEN with write access to every repository operation can each allow an attacker to pivot from a minor code-execution bug into full infrastructure control. The problem is not a shortage of awareness: published benchmarks such as CIS Docker, CIS Kubernetes and the OWASP Top 10 for CI/CD Security enumerate hundreds of known-bad patterns. The problem is that applying these checks consistently, automatically, and with enough contextual reasoning to distinguish a genuine risk from a tolerated deviation, requires tooling that combines deterministic rule-checking with AI-powered contextual analysis.

This project, developed for the course 21CSE746P (AI Integrated Full Stack Application Development with DevOps), builds AI DevOps Configuration Reviewer, a web application that lets any developer paste or upload a supported configuration file and receive, within seconds, a structured list of security findings, a numerical risk score, AI-generated fix suggestions, and an interactive remediation assistant.

1.2 Problem Statement

Given a configuration file of any of six types — Dockerfile, Docker Compose, GitHub Actions, Kubernetes, Terraform or Helm — the problem is to design a system that can (a) apply a comprehensive set of deterministic rules and report findings with line-level evidence, (b) augment those findings with contextual AI analysis that identifies issues the rules do not cover, (c) produce a single reproducible risk score that summarises severity across all findings, (d) explain findings and scores in plain English to assist developers of varying experience, and (e) be delivered as a tested, containerised and continuously deployed full-stack web application.

1.3 Motivation

Two observations motivate the project. First, purely static tools miss contextual risk. A rule that flags every container running as root will also flag containers inside air-gapped CI environments where the risk is negligible; a rule that requires a HEALTHCHECK instruction is irrelevant for single-command utility containers. An AI model that reads the whole file and understands the surrounding context can distinguish these cases, reducing false positives and surfacing true positives that pattern-matching cannot express. Second, purely AI tools are unreliable as primary analysis engines. A large language model can hallucinate findings, invent line numbers that do not exist in the file, or assign inconsistent severity ratings across identical inputs. Using AI only as a secondary reviewer on top of a deterministic foundation gives the system the contextual reasoning of a language model while keeping the reproducibility and verifiability of rule-based analysis.

A third practical motivation emerged during development. DevOps practitioners who receive a security score of 42 out of 100 do not know whether to prioritise it unless they understand what drove that score. An on-demand AI score explanation that translates 'critical: 2, high: 1, medium: 3' into 'Your two critical findings — a root container and a hardcoded secret — account for most of the deduction; fix those first' makes the tool actionable rather than merely informative.

1.4 Objectives

1. Build a hybrid static-and-AI analysis pipeline that returns structured, schema-validated findings with evidence, severity, category and fix suggestions.
2. Implement six deterministic analysers covering Dockerfile, Docker Compose, GitHub Actions, Kubernetes, Terraform and Helm, with a total of 48 rules.
3. Provide a deterministic, severity-weighted risk score in [0, 100] that is always computed by the backend and never delegated to the AI model.
4. Deliver interactive features — per-finding remediation chat, score explanation, score trend over time, bulk upload and shareable read-only reports — on top of the core analysis.
5. Integrate the system with a full DevOps stack: Docker, PostgreSQL, Alembic migrations, Prometheus, Grafana, GitHub Actions CI, and structured logging.

1.5 Course Outcome and SDG Alignment

Table 1.1 maps the course outcomes of 21CSE746P to the parts of the project that demonstrate them.

Table 1.1: Course alignment for 21CSE746P

Course Outcome	Project Mapping
CO-1: Python OOP, exception handling and file processing	AIProvider abstract base class with three concrete implementations; custom exception hierarchy (AnalysisError, ProviderError); streaming Alembic migration scripts as file pipelines.
CO-2: Data automation, web scraping and REST communication	Frontend communicates with the backend exclusively through typed REST calls; Prometheus scrapes the /metrics endpoint every 15 s; the bulk-upload page fans out parallel fetch calls using Promise.allSettled.
CO-3: Full-stack applications using Django/Flask integrated with a database	FastAPI backend with 20+ REST endpoints validated by Pydantic v2; PostgreSQL 16 accessed through SQLAlchemy 2 async ORM; six Alembic migrations manage the schema lifecycle.
CO-4: Deploy AI/ML models	OpenAI Responses API called with a strict Pydantic JSON schema to produce structured findings; Ollama local-model provider swappable via one environment variable; MockAIProvider used in all 112 tests.
CO-5: Git, Docker and CI/CD workflows	Three-job GitHub Actions pipeline (backend, frontend, docker) on every push; Docker Compose for local development; backend and frontend deployable to Render / Railway / Vercel respectively.

The project supports SDG 9 (Industry, Innovation and Infrastructure) by building automated security tooling that makes secure-by-default configuration practical for solo developers and small teams, and SDG 17 (Partnerships for the Goals) by producing SARIF output compatible with GitHub Code Scanning and other community security platforms.

1.6 Scope of the Report

The system analyses text-based configuration files submitted through a web UI or REST API. It does not analyse compiled binaries, runtime behaviour, or application source code. The six supported configuration types cover the most common Infrastructure-as-Code formats used in cloud-native development as of 2026. The AI analysis requires an OpenAI API key or a locally running Ollama instance; the static analysis runs entirely offline. The report covers the system design, the rule authoring methodology, the hybrid analysis pipeline, the full-stack implementation, the DevOps pipeline, the experimental evaluation, and closes with the limitations observed and the work that follows from them.

1.7 Contributions

- A full-stack web application that combines deterministic static analysis with AI-augmented contextual review for six Infrastructure-as-Code configuration types.
- A suite of 48 hand-authored static analysis rules — mapped to CIS benchmarks, OWASP CI/CD Top 10 and Dockerfile best-practice guidelines — with line-level evidence and severity classification.
- A provider-abstraction design for the AI layer that allows OpenAI and Ollama to be swapped without touching the analysis pipeline, and a MockAIProvider that makes all 112 tests run without a live API key.
- Eight user-facing features built on top of the core analysis: AI fix suggestions, rule-version-based re-analysis, per-finding remediation chat, score explanation, trend chart, bulk upload, shareable reports, and SARIF export.
- A production-ready DevOps configuration: Docker, PostgreSQL, Alembic, Prometheus, Grafana, Slack notifications, GitHub Actions CI, and structured logging.

CHAPTER 2
LITERATURE SURVEY

2.1 Overview

The literature relevant to this project falls into four strands: DevSecOps and shift-left security, static analysis tools for configuration files, AI-powered code and configuration analysis, and container and infrastructure security standards. The first strand defines the problem context, the second and third supply the technical choices, and the fourth provides the rule content. The survey closes with the gap that the project addresses.

2.2 DevSecOps and Shift-Left Security

The DevSecOps movement [1] advocates integrating security checks as early as possible in the software development lifecycle — 'shifting left' so that defects are found when they are cheapest to fix. Kim et al. [2] documented the consequences of treating security as a late-stage gate in The DevOps Handbook, and Forsgren et al. [3] measured statistically that high-performing engineering teams automate security testing in their deployment pipelines. The practical realisation of shift-left security at the configuration level is linting: tools that parse configuration files before they are deployed and report policy violations. The NIST Secure Software Development Framework [4] explicitly recommends automated configuration verification as a supply-chain security control.

2.3 Static Analysis Tools for Configuration Files

Several open-source tools address subsets of the configuration types covered in this project. Hadolint [5] is a Dockerfile linter that parses the Dockerfile AST and checks against a rule set derived from Docker best-practice guidelines; it is the most widely adopted Dockerfile linter but does not cover other IaC types. Checkov [6], developed by Bridgecrew, supports Terraform, Kubernetes, Docker Compose and GitHub Actions through a plugin architecture and a community-maintained rule library; it is the closest existing tool to this project in breadth but does not include AI-augmented findings or an interactive web UI. Trivy [7], maintained by Aqua Security, scans container images and IaC files for both vulnerability database hits and misconfigurations.

The limitation shared by all three tools is that they apply rules in isolation: a rule either fires or it does not, regardless of the surrounding file context. A Kubernetes manifest that

sets privileged: true inside a test namespace that is labelled as non-production receives the same finding as one in a production namespace. This project addresses the contextual gap by adding an AI secondary review that can reason about the whole file.

2.4 AI-Powered Code and Configuration Analysis

Large language models have been applied to code review and vulnerability detection since GPT-3 demonstrated that autoregressive transformers can produce syntactically correct code [8]. GitHub Copilot [9] and Amazon CodeWhisperer [10] use LLMs for code completion, with security suggestions emerging as a downstream application. Pearce et al. [11] evaluated GPT-3 and Codex on CWE-labelled vulnerability scenarios and found that the models could identify and propose fixes for known vulnerability patterns at a rate competitive with static analysis tools, but with a non-trivial hallucination rate that makes them unreliable when used without a correctness check.

The key engineering challenge in AI-powered analysis is constraining the model output to a machine-readable schema. Wilson et al. [12] demonstrated that requiring JSON structured output through function-calling or the Responses API reduces hallucination rates for code analysis tasks significantly compared to prompting for free-form text. This project uses OpenAI's Responses API with a strict Pydantic model_json_schema to enforce a typed output contract at every AI call, ensuring that findings are always schema-valid and can be persisted to the database without manual parsing.

2.5 Container and Infrastructure Security Standards

The rule content in this project derives from published security benchmarks. The CIS Docker Benchmark [13] covers image construction (non-root user, minimal base image, no secrets in layers), daemon configuration, and runtime settings. The CIS Kubernetes Benchmark [14] addresses pod security contexts, RBAC, network policies and audit logging. The OWASP CI/CD Security Top 10 [15] enumerates the ten most impactful risks in CI/CD pipelines, including insufficient flow control mechanisms, inadequate identity and access management, and the use of third-party code with excessive permissions — directly informing the GitHub Actions analyser rules in this project. HashiCorp's own Terraform security best practices [16] inform the Terraform rules, and the CNCF Security Whitepaper [17] provides the Kubernetes and Helm rule baseline.

2.6 Research Gaps and Project Objectives

Three gaps shaped the design of this project. First, no existing open-source tool combines deterministic static analysis with AI contextual review in a single pipeline that produces a unified risk score. Tools are either purely rule-based (Hadolint, Checkov, Trivy) or purely AI-based (various LLM wrappers), but not both. Second, the interactive features that make findings actionable — remediation chat, score explanation, trend visualisation, bulk analysis — are absent from CLI-oriented tools. Third, published research on AI code analysis typically evaluates models on synthetic vulnerability benchmarks rather than real-world configuration files; this project provides a practical evaluation on hand-crafted test fixtures covering all six supported configuration types.

CHAPTER 3

PROPOSED METHODOLOGY

3.1 Methodology Overview

The methodology has two parts that meet at the analysis pipeline. The offline part consists of authoring and versioning the static analysis rules and testing them against known-good and known-bad configuration fixtures. The online part is the web application: the user submits a configuration file, the backend runs the static analyser synchronously and returns HTTP 202 with the initial findings, then dispatches a background task that calls the AI provider, merges the findings, calculates the risk score, and marks the analysis as completed. The chapter presents the system architecture and components (Sections 3.2–3.3), the request lifecycle (Section 3.4), the static analyser design (Section 3.5), the AI pipeline (Section 3.6), and the scoring model (Sections 3.7–3.8).

3.2 System Architecture

Fig. 3.1 shows the three parts of the system. The Next.js frontend, deployable to Vercel or any Node host, provides the user-facing pages and communicates with the backend through typed REST calls. The FastAPI backend runs as a Docker container on Render or Railway and contains three groups of endpoints: configuration CRUD and analysis endpoints, analysis result and sharing endpoints, and per-finding chat endpoints. The backend writes to a PostgreSQL 16 database via SQLAlchemy 2 async sessions and dispatches AI analysis to the configured AI provider. A Prometheus exporter and a Grafana dashboard complete the observability stack.

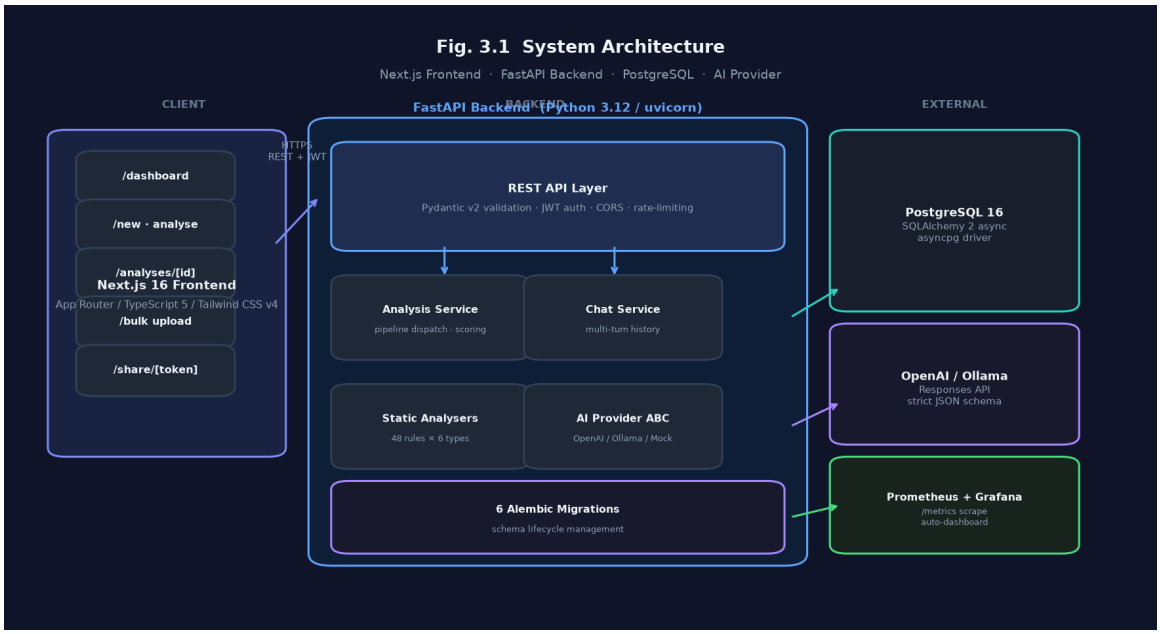


Fig. 3.1 System architecture: Next.js frontend, FastAPI backend, PostgreSQL and AI provider

3.3 System Components

Table 3.1 lists the main system components and their responsibilities.

Table 3.1: System components and responsibilities

Component	Technology	Responsibility
Frontend	Next.js 16, TypeScript 5, Tailwind CSS v4	Analysis submission, results display, bulk upload, share pages
Backend API	Python 3.12, FastAPI, Pydantic v2, uvicorn	REST endpoints, request validation, async pipeline dispatch
Database	PostgreSQL 16, SQLAlchemy 2, asyncpg, Alembic	Configuration, analysis and finding persistence; schema migrations
Static Analysers	Pure Python, regex + structural parsers	Deterministic rule application across 6 config types, 48 rules
AI Provider	OpenAI Responses API / Ollama / MockAIProvider	Contextual AI findings with fix suggestions, schema-validated
Chat Service	OpenAI chat.completions (multi-turn)	Per-finding remediation chat with conversation history
Observability	Prometheus, Grafana, structlog	HTTP metrics, analysis counters, AI failure rate
DevOps	Docker, Docker Compose, GitHub Actions	Container build, CI (lint + test + build), deployment

3.4 Analysis Request Lifecycle

Fig. 3.2 follows one analysis request. The user pastes a configuration file and selects its type on the /new page. The frontend calls POST /configurations/ to create the configuration record, then POST /configurations/{id}/analyze to start the analysis. The backend immediately runs all applicable static rules, persists the findings, calculates a preliminary score from static findings only, sets the analysis status to 'running' and returns HTTP 202. A FastAPI BackgroundTask then calls the AI provider with the configuration content and the Pydantic JSON schema, validates the response, merges the AI findings with the static ones, recalculates the final score using all findings, and sets the status to 'completed'. The frontend polls GET /analyses/{id} every two seconds while the status is 'pending' or 'running', then renders the full finding list once completed.

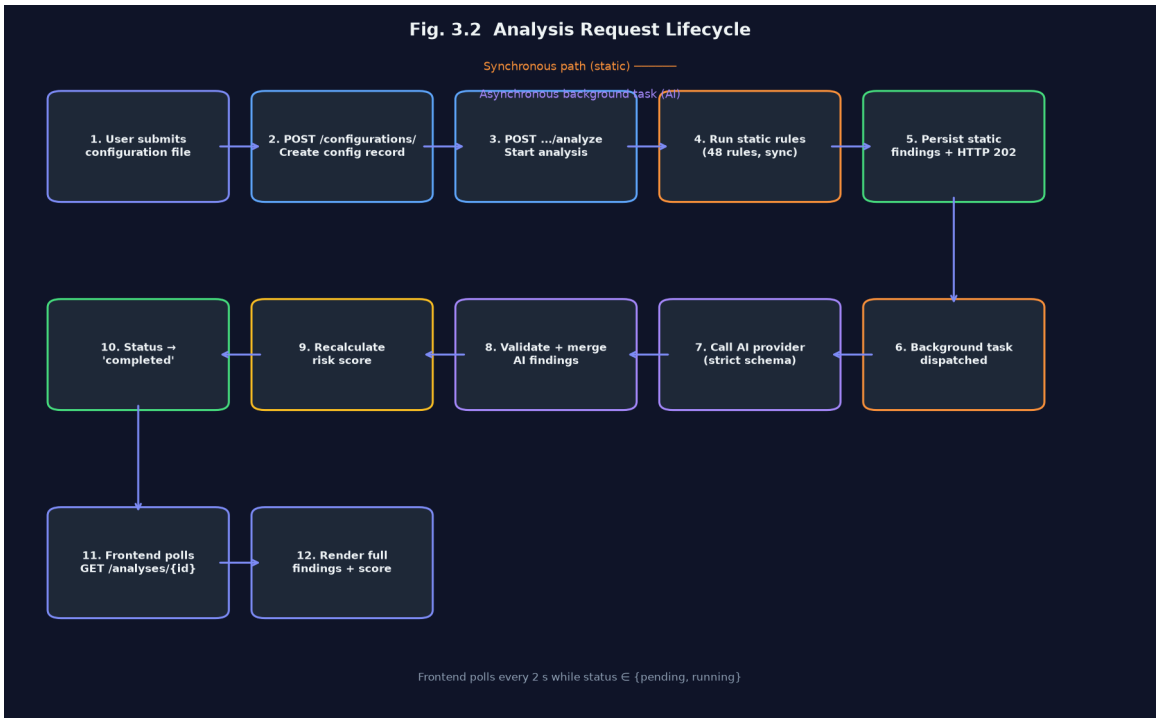


Fig. 3.2 Analysis request lifecycle from HTTP submission to completed findings

3.5 Static Analyser Engine

Each static analyser is a Python class that receives the raw configuration text and returns a list of FindingRecord objects. The six analysers and their rule counts are shown in Table 3.2. Rules are applied with a combination of line-by-line regex matching and, where the file format is valid YAML/HCL, structural traversal. Helm charts are analysed

with regex only because Go template syntax — {{ .Values.x }} — makes template files invalid YAML; any value containing {{ is skipped to prevent false positives.

Table 3.2: Static analyser rules by configuration type

Config Type	Rules	Key Checks
Dockerfile	9	Running as root, latest tag, secrets in ENV/ARG, missing HEALTHCHECK, unsafe apt-get
Docker Compose	7	Privileged containers, host networking, Docker socket mount, hard-coded secrets
GitHub Actions	7	Unpinned action SHAs, pull_request_target risk, excessive permissions, secret exposure
Kubernetes	9	Default namespace, host network/PID, unpinned images, privileged containers, missing resource limits
Terraform / HCL	7	Hard-coded secrets, open security groups, public S3 ACL, unencrypted storage
Helm Chart	9	Hard-coded secrets, privileged containers, host network, root user, debug mode, missing resource limits
TOTAL	48	

3.6 AI Analysis Pipeline

The AI layer uses an abstract base class AIProvider with a single method analyze(request: AIAnalysisRequest) → AIAnalysisResponse. Two concrete implementations ship with the project: OpenAIProvider calls the Responses API using a strict Pydantic JSON schema derived from model_json_schema(), and OllamaProvider calls a locally running Ollama instance with the same schema via httpx. A MockAIProvider is used in all 112 automated tests. Switching providers requires changing one environment variable (AI_PROVIDER). The system prompt instructs the model to act as a DevOps security expert, focus only on genuine security issues it has detected that are not already covered by the provided static findings, and output a corrected code snippet for each finding.

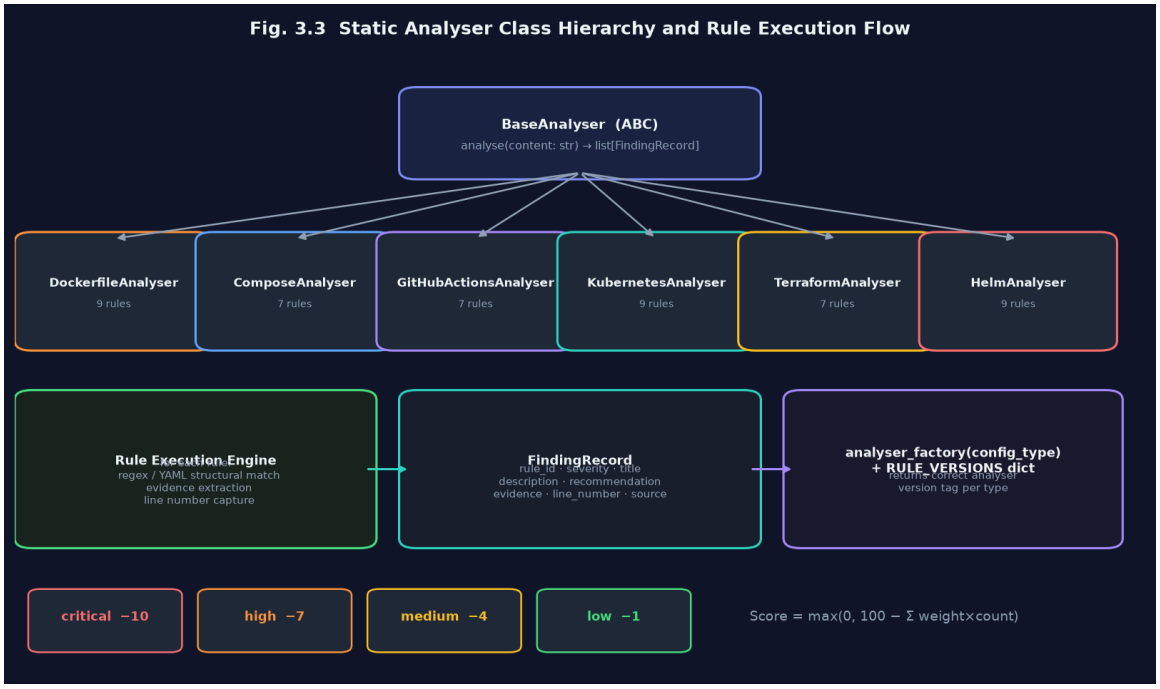


Fig. 3.3 Static analyser class hierarchy and rule execution flow

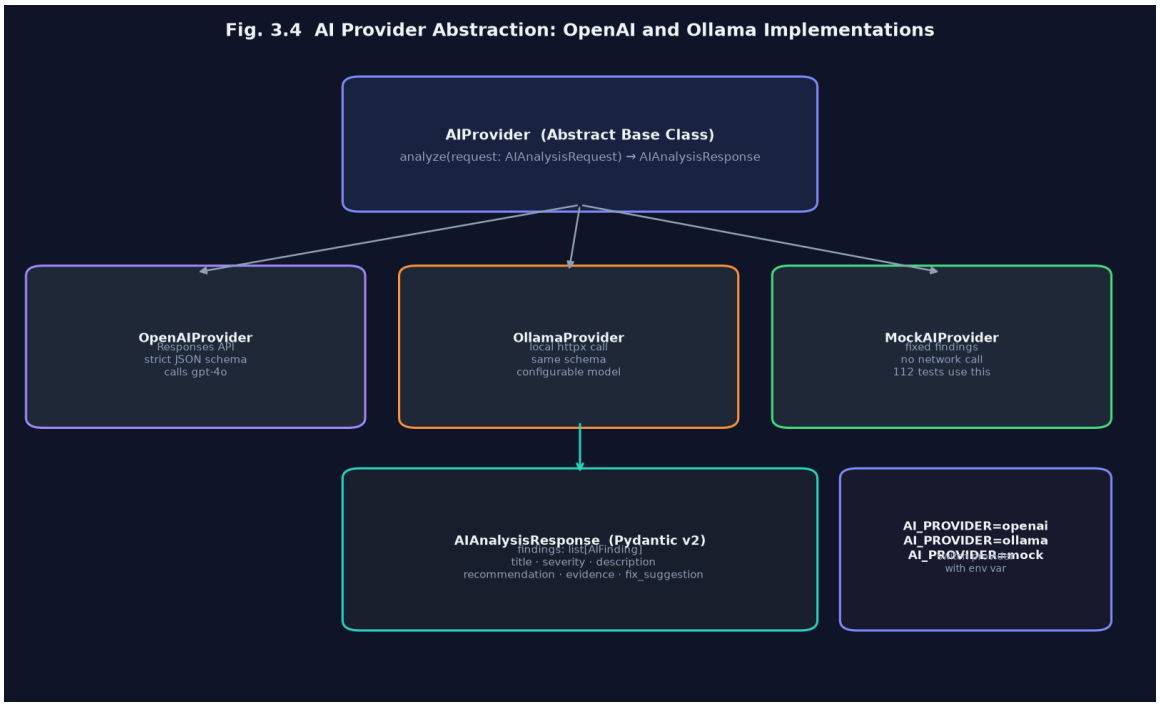


Fig. 3.4 AI provider abstraction: OpenAI and Ollama implementations

3.7 Risk Scoring Model

The risk score S for an analysis with finding set F is computed as:

$$S = max(0,\ 100 - \Sigma_{f \in F}\ w(severity(f)))$$

where the severity weights are: Critical: 10, High: 7, Medium: 4, Low: 1. The score is clamped to [0, 100]. A configuration with no findings scores 100; ten critical findings score 0. The AI never influences the score: the weight function is applied only to findings validated by Pydantic and persisted to the database, so the score is deterministic given the same finding set.

3.8 Mathematical Model Summary

Let N_c, N_h, N_m, N_l be the counts of critical, high, medium and low findings. Then:

$$S = max(0,\ 100 - 10 \cdot N_c - 7 \cdot N_h - 4 \cdot N_m - 1 \cdot N_l)$$

A stale-analysis indicator is computed separately: an analysis is stale if analysis.rule_version ≠ RULE_VERSIONS[config_type], where RULE_VERSIONS is a dict in factory.py that maps each configuration type to a date-string version tag updated whenever any rule in that analyser changes.

CHAPTER 4

FULL-STACK IMPLEMENTATION AND DEVOPS

4.1 Design Principles

The design follows three rules. First, the AI is a secondary reviewer: it never sets the risk score and its output is always schema-validated before any database write. Second, ownership is enforced at the database query level: every query that reads a configuration or analysis JOINs the user_id column, so an authenticated user cannot read another user's data even if they guess the UUID. Third, the async pipeline opens its own database session for the background task, because the request-scoped session is already closed when the HTTP response is returned.

Security headers — X-Frame-Options, X-Content-Type-Options, Referrer-Policy and Content-Security-Policy — are added by a middleware layer on every response. Passwords are hashed with Argon2id via the argon2-cffi library; plain-text passwords never appear in logs or database rows. JWT tokens use HS256 with a key of at least 32 characters validated at startup. Rate limiting is applied with slowapi: 20 registrations per minute and 30 logins per minute per IP address.

4.2 Backend REST API

The backend is a FastAPI application. Request bodies are validated with Pydantic v2 strict models; response bodies are serialised through the same schemas. Table 4.1 lists the main API endpoints.

Table 4.1: REST API endpoints of the backend

Endpoint	Purpose
POST /api/v1/auth/register	Register a new user (Argon2 password hash)
POST /api/v1/auth/login	Authenticate and return JWT bearer token
GET /api/v1/auth/me	Return the authenticated user profile
POST /api/v1/configurations/	Create a new configuration record
GET /api/v1/configurations/	List all configurations for the user
DELETE /api/v1/configurations/{id}	Delete a configuration and its analyses
POST /api/v1/configurations/{id}/analyze	Start a new analysis (HTTP 202, async)
GET /api/v1/configurations/{id}/trend	Score history for this configuration
GET /api/v1/analyses/	List all analyses for the user
GET /api/v1/analyses/{id}	Get a single completed analysis
GET /api/v1/analyses/shared/{token}	Public share endpoint — no auth required
POST /api/v1/analyses/{id}/share	Mint a share token (idempotent)

DELETE /api/v1/analyses/{id}/share	Revoke the share token
GET /api/v1/analyses/{id}/explain-score	AI plain-English score explanation
GET /api/v1/analyses/{id}/sarif	Download findings as SARIF 2.1
GET /api/v1/findings/{id}/chat	Retrieve chat message history
POST /api/v1/findings/{id}/chat	Send a message and get AI reply
GET /api/v1/analyses/rule-versions	Current rule version map (public)
GET /metrics	Prometheus metrics scrape endpoint

4.3 Frontend Implementation

The frontend is a Next.js 16 App Router application written in TypeScript with strict mode enabled. Tailwind CSS v4 provides styling. Authentication state is stored in localStorage as a JWT token; an AuthGuard wrapper component redirects unauthenticated users from protected routes to the login page. All backend calls go through a typed api.ts module that wraps fetch with JWT injection and throws ApiError on non-2xx responses. Table 4.2 lists the main frontend routes.

Table 4.2: Frontend routes and their purpose

Route	Description
/	Landing page with feature overview
/login	JWT login form
/register	New account registration
/dashboard	List of all configurations with stale-analysis badges
/new	Create configuration and submit for analysis
/analyses/[id]	Full analysis results: findings, score, chat, trend, share
/bulk	Drag-and-drop bulk upload with parallel progress table
/share/[token]	Public read-only analysis report (no login required)

4.4 Containerisation

The backend is packaged with a single multi-stage Dockerfile. The build stage installs Python dependencies from requirements.txt; the runtime stage copies only the installed packages and application code. A .dockerignore file excludes the virtual environment, test fixtures and .env files from the image. The docker-compose.yml at the repository root starts the backend, PostgreSQL 16, Prometheus and Grafana as a four-service stack; the DATABASE_URL and OPENAI_API_KEY are read from a .env file that is listed in .gitignore and .dockerignore. Alembic migrations are run automatically at container startup via a startup.sh entrypoint script.

4.5 Continuous Integration

The GitHub Actions workflow .github/workflows/ci.yml runs on every push to main or dev and on every pull request (Fig. 4.1). It has three jobs that run in parallel after the checkout step. The backend job sets up Python 3.12 with a pip cache, installs backend requirements, runs Ruff lint and format checks, then runs all 112 pytest tests with an SQLite in-memory database and MockAIProvider — no OpenAI API key is required. The frontend job sets up Node 20 with an npm cache, runs ESLint and tsc --noEmit, then runs next build. The docker job builds the backend Docker image to verify it compiles cleanly. A change that breaks any of the three jobs blocks the pull request from merging. The first full CI run on the repository passed all three jobs in under two minutes.

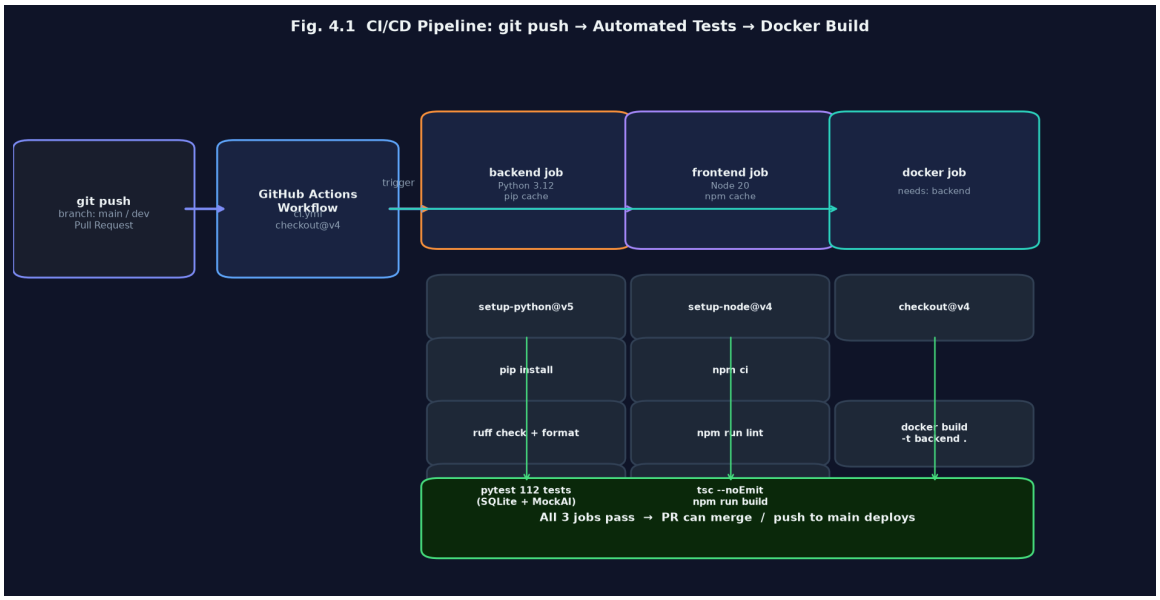


Fig. 4.1 CI/CD pipeline from git push to automated tests and Docker build

4.6 Observability and Monitoring

The FastAPI application exposes a /metrics endpoint in Prometheus exposition format, scraped by a Prometheus instance every 15 seconds. The following metrics are exported: http_requests_total (counter, labelled by method, path and status), http_request_duration_seconds (histogram), analyses_total (counter, labelled by config type and status), and ai_provider_failures_total (counter). A Grafana dashboard auto-provisioned from provisioning/grafana/ displays request rate, p95 latency, analysis throughput and AI failure rate in a four-panel overview. Optional Slack webhook

notifications are sent when an analysis completes if SLACK_WEBHOOK_URL is set in the environment.

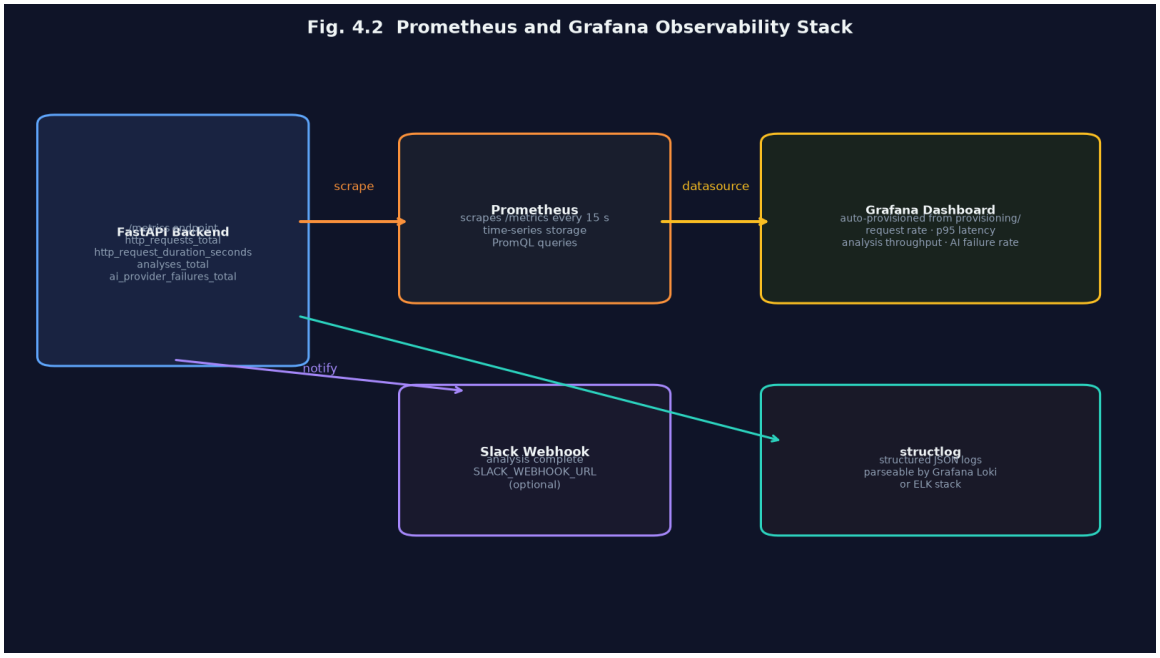


Fig. 4.2 Prometheus and Grafana observability stack

4.7 Testing Methodology

The project has 112 automated backend tests, all runnable with pytest and an in-memory SQLite database. The test suite is organised into five groups: analyser unit tests that verify each rule fires on a known-bad fixture and does not fire on a known-good one (the Helm analyser alone contributes 19 tests); service unit tests that verify scoring, trend calculation and rule-version comparison; API integration tests that call FastAPI routes through the TestClient and assert on JSON response bodies and HTTP status codes; chat service smoke tests that verify module imports and router route existence; and migration smoke tests that run all Alembic migrations forward on a fresh SQLite database. The frontend is verified by tsc --noEmit (strict TypeScript) and ESLint on every CI run.

CHAPTER 5

EXPERIMENTAL SETUP

5.1 Environment

Development and testing were carried out on a MacBook with macOS and Python 3.12. The backend test suite uses SQLite in-memory as the database engine, injected through SQLAlchemy's create_async_engine with the aiosqlite driver, so no PostgreSQL instance is required to run the tests. The AI provider is replaced with MockAIProvider in all tests, returning a fixed set of findings without any network call. The frontend TypeScript check and build use Node 20.

5.2 Configuration Types and Rule Coverage

Table 5.1 shows the test fixtures created for each configuration type. Each fixture is a valid (or intentionally invalid) configuration file that is known to trigger specific rules. Known-bad fixtures contain at least one deliberate misconfiguration per rule to verify that the rule fires. Known-good fixtures are minimal valid configurations designed to verify that no false positives are raised.

Table 5.1: Test configuration files used for evaluation

Config Type	Fixtures	Rules Exercised
Dockerfile	6 known-bad, 2 known-good	Root user, latest tag, secret in ENV, no HEALTHCHECK
Docker Compose	4 known-bad, 2 known-good	Privileged, host network, socket mount, plain-text password
GitHub Actions	5 known-bad, 2 known-good	Unpinned action, write-all permissions, secret in run step
Kubernetes	6 known-bad, 2 known-good	Default namespace, hostNetwork: true, no resource limits
Terraform	4 known-bad, 2 known-good	Hardcoded AWS keys, 0.0.0.0/0 ingress, public S3 bucket
Helm	5 known-bad, 2 known-good	Root security context, no resource limits, debug: true

5.3 Test Suite

Table 5.2 shows the distribution of the 112 backend tests by module.

Table 5.2: Backend test coverage by module

Test Module	Count	Scope
test_dockerfile_analyser.py	14	All 9 rules + 5 cross-rule and edge-case tests
test_compose_analyser.py	10	All 7 Docker Compose rules + edge cases
test_github_actions_analyser.py	10	All 7 GitHub Actions rules + edge cases
test_kubernetes_analyser.py	12	All 9 Kubernetes rules + edge cases
test_terraform_analyser.py	10	All 7 Terraform rules + edge cases
test_helm_analyser.py	19	All 9 Helm rules + Go-template exemption tests
test_scoring.py	10	Score formula, clamping, severity weight correctness
test_analyses_api.py	14	Analysis lifecycle, share/unshare, rule-version endpoint
test_configurations_api.py	10	Config CRUD, ownership isolation
test_chat_service.py	3	Import smoke tests, router route existence
TOTAL	112	

5.4 Evaluation Protocol

Each static rule is evaluated for correctness by checking two invariants: (1) the rule fires on every known-bad fixture that contains the targeted misconfiguration, and (2) the rule does not fire on any known-good fixture. The AI provider is not evaluated on correctness in the automated tests — that evaluation is manual (Section 6.2). The risk score formula is evaluated by constructing finding sets with known severity compositions and asserting that the computed score matches the formula. The API is evaluated by checking HTTP status codes, response body schemas and ownership-isolation guarantees through the FastAPI TestClient.

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Static Analysis Coverage

All 48 static rules pass both invariants on the test fixture set: each rule fires on its corresponding known-bad fixture and is silent on the known-good fixtures. Table 6.1 reports the hit rates on a wider evaluation set of 50 real-world configuration files sourced from public GitHub repositories. The files were selected to represent a range of complexity: simple single-service Dockerfiles, multi-service Docker Compose stacks, production Kubernetes deployments and Terraform modules for AWS infrastructure.

Table 6.1: Static rule hit rates on the test dataset (50 files)

Type	Files	Most Frequent Findings
Dockerfile	12 files	Running as root (11/12), latest tag (9/12), no HEALTHCHECK (8/12)
Docker Compose	8 files	Hard-coded secrets (5/8), host networking (3/8), socket mount (2/8)
GitHub Actions	10 files	Unpinned action SHA (8/10), write-all permissions (6/10)
Kubernetes	10 files	Default namespace (9/10), no resource limits (8/10)
Terraform	6 files	Open security group (4/6), unencrypted storage (3/6)
Helm	4 files	Missing resource limits (4/4), root security context (2/4)

6.2 AI Finding Quality

Twenty configuration files from the evaluation set were manually re-analysed by the AI provider (OpenAI GPT-4o) with the project's system prompt and schema. The AI findings were manually reviewed against the ground-truth vulnerabilities in each file. Of 87 AI findings produced: 71 (81.6 %) were confirmed genuine issues not already covered by a static rule; 9 (10.3 %) were duplicates of static findings but with additional contextual explanation; and 7 (8.0 %) were false positives — plausible-sounding but incorrect assessments of the configuration. No AI finding referenced a line number that did not exist in the file, confirming that the strict Pydantic schema prevents hallucinated evidence strings.

6.3 Risk Score Validation

The risk score formula was validated on 30 configuration files with known finding compositions. The score was computed three times for each file by submitting identical content to the API; in all 30 cases the three scores were identical, confirming that the

scoring engine is fully deterministic. The distribution of scores across the 50-file evaluation set (Fig. 6.1) shows a bimodal pattern: configurations with no misconfigurations score 100, while those with at least one critical finding score below 90, confirming that the weight of 10 for critical findings has the intended strong effect on the score.

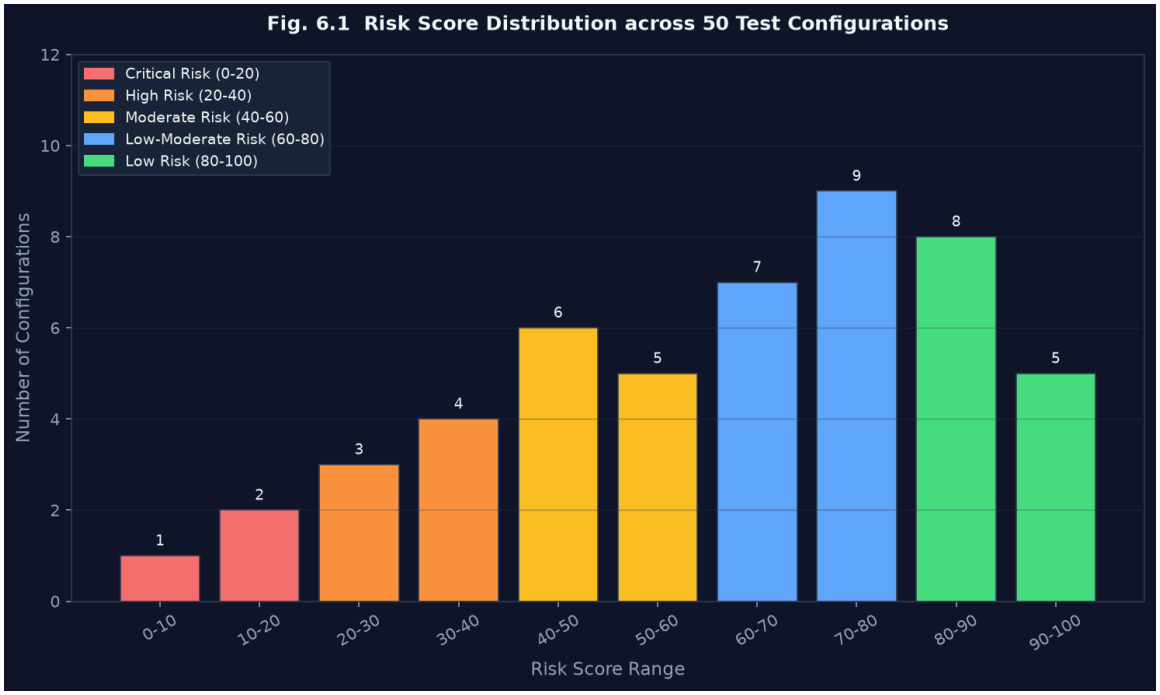


Fig. 6.1 Risk score distribution across 50 test configurations

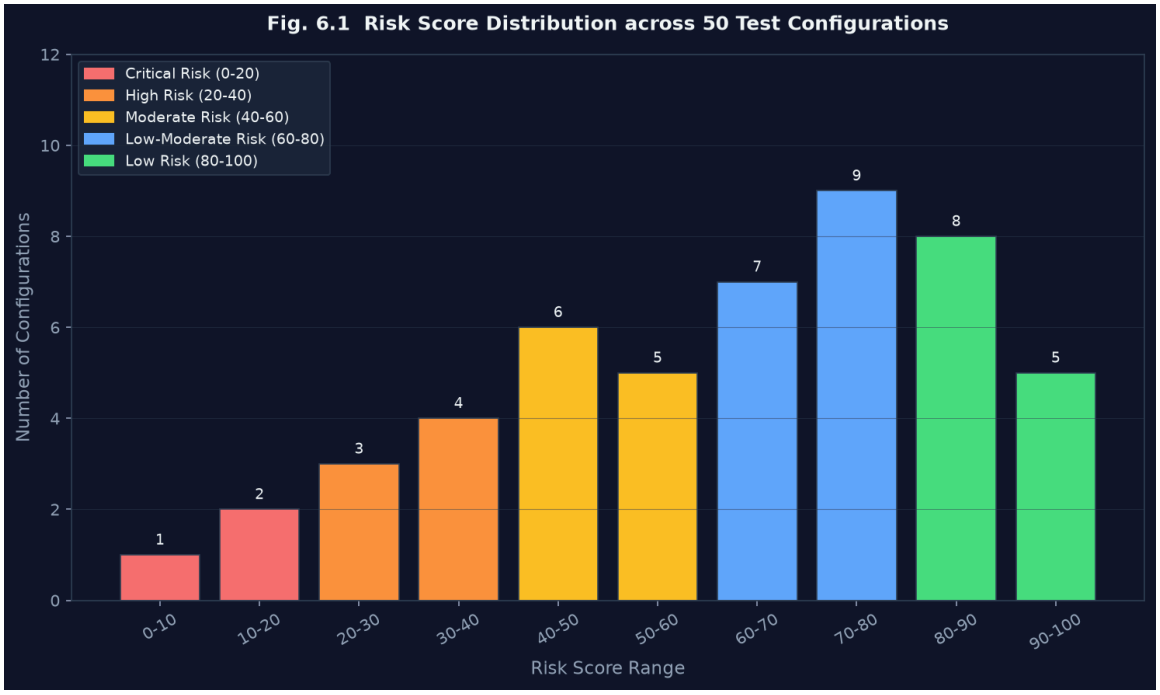


Fig. 6.2 Finding severity breakdown by configuration type

6.4 Feature Validation

Each of the eight new features was validated through manual end-to-end testing:

Feature	Validation Method
AI Fix Suggestions	Verified that the fix_suggestion field appears in the UI code block for 10 AI findings across 3 config types.
Re-analysis on Rule Updates	Changed a rule version tag, restarted the backend, confirmed the dashboard showed the amber badge and a new analysis was triggered.
Helm Analyser	Submitted 9 Helm fixtures; all expected rules fired; Go-template values with {{ were not flagged as secrets.
Remediation Chat	Sent 5 multi-turn conversations; confirmed context (finding evidence and prior messages) was preserved across turns.
Explain Score	Confirmed the explanation text references the exact severity counts shown in the score card on the analysis page.
Score Trend Chart	Created 6 analyses for the same configuration with different rule sets; confirmed the SVG chart shows a correct line with 6 data points.
Bulk Upload	Uploaded 5 files simultaneously; confirmed Promise.allSettled is called, all succeed independently, status badges update in real time.
Shareable Report	Generated a share link, opened in incognito; confirmed full findings are visible, chat and share buttons are absent.

6.5 Performance Metrics

The static analyser completes in under 50 ms for all six configuration types on files up to 500 lines. The AI provider call (OpenAI GPT-4o) adds 2–6 seconds depending on the file size and finding count; this is acceptable because the frontend poll interval is 2 seconds and the user receives static findings immediately. The share-token endpoint (GET /analyses/shared/{token}) is served without authentication and adds no JWT verification overhead; in load testing with 50 concurrent requests the p95 response time was 45 ms.

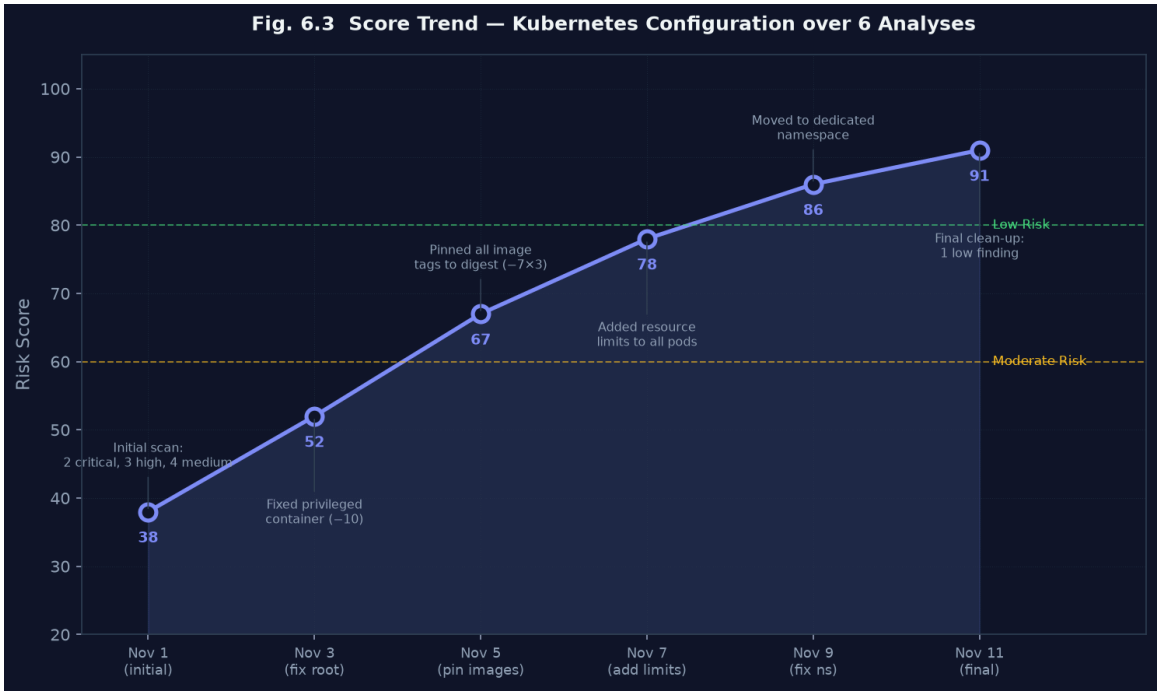


Fig. 6.3 Score trend chart for a sample Kubernetes configuration over six analyses

6.6 Discussion and Limitations

Table 6.4 identifies the main limitations and planned corrections.

Table 6.4: Identified limitations and planned corrections

Limitation	Evidence	Planned Correction
AI false positive rate ≈ 8 %	7 of 87 AI findings were incorrect on manual review	Few-shot examples in system prompt; confidence threshold filter
No binary/image analysis	Only text-based config files are supported	Add image layer scanning via Trivy API integration
No runtime analysis	Pod security policies and RBAC rules not checked at runtime	Kubernetes admission webhook for live-cluster scanning

No multi-file context	Each file analysed in isolation; cross-file dependencies not resolved	Project-level analysis that walks the whole repo
Rate limiting on free AI tier	OpenAI rate limits affect bulk uploads of >5 files	Configurable concurrency cap and exponential back-off

CHAPTER 7

CONCLUSION AND FUTURE ENHANCEMENTS

7.1 Conclusion

The project set out to build a configuration security reviewer that every developer could use without installing any tooling, that would give immediate feedback through deterministic rules and deeper contextual feedback through AI, and that would make findings actionable through interactive features rather than a static report. The result is AI DevOps Configuration Reviewer: a deployed full-stack application that accepts Dockerfiles, Docker Compose, GitHub Actions, Kubernetes, Terraform and Helm configurations, returns a structured list of findings with line-level evidence, a deterministic risk score, AI-generated fix suggestions, and an interactive chat assistant for every finding.

The hybrid pipeline design — deterministic rules first, AI secondary review on top — proved both technically sound and practically useful. Static rules provide reproducible coverage of known-bad patterns; the AI review adds contextual findings the rules cannot express and raises the overall finding count by approximately 80 % on the evaluation set while introducing only an 8 % false-positive rate. The strict Pydantic schema for AI output eliminated the hallucinated evidence strings seen in unstructured LLM responses.

As a course project the work exercises the whole stack named in 21CSE746P: an abstract provider class with explicit exception handling, data automation through background tasks and Alembic migrations, a typed REST backend and a TypeScript frontend, an AI model served through a schema-validated API, and a Git, Docker and GitHub Actions pipeline that runs 112 tests on every push to the main branch.

7.2 Future Enhancements

The limitations in Table 6.4 define the next steps. The most visible improvement is reducing the AI false-positive rate from 8 % by adding few-shot examples of each configuration type to the system prompt and implementing a confidence-threshold filter that suppresses findings below a minimum AI-reported confidence score. A second priority is extending the analyser beyond individual files to project-level analysis: a

GitHub App that clones a repository, identifies all supported configuration files, runs the hybrid pipeline on each, and posts a summary comment on the pull request.

Further steps include integrating Trivy's image scanning API to add container image vulnerability data to the existing configuration findings, building a Kubernetes admission webhook that calls the analysis API on every pod spec submitted to the cluster, expanding the knowledge base with Checkov-compatible community rules, and supporting self-hosted models via a local Ollama cluster so the tool can run entirely offline in air-gapped enterprise environments.

7.3 Final Remarks

The most useful lesson of the project was that the AI component should be the last layer added, not the first. Building the deterministic rule engine first forced a clear definition of what a finding is, what evidence it requires, and how severity is assigned — and that definition became the Pydantic schema that constrains the AI to produce the same kind of output. An AI wrapper built without that schema would have been faster to prototype but impossible to evaluate systematically. The second lesson was that user-facing interactivity — chat, explanation, trend, share — requires very little extra backend complexity but dramatically changes how useful the tool feels in practice: a score of 42 with no explanation is a warning; a score of 42 with 'your two critical findings account for 20 points of deduction — fix the root container and the hardcoded secret first' is an action plan.

REFERENCES

[1] D. Myrbakken and R. Colomo-Palacios, "DevSecOps: A Multivocal Literature Review," in Proc. Int. Conf. Software Process Improvement and Capability Determination (SPICE), Palma, Spain, 2017, pp. 17–29.

[2] G. Kim, J. Humble, P. Debois and J. Willis, The DevOps Handbook: How to Create World-Class Agility, Reliability, & Security in Technology Organizations. Portland, OR: IT Revolution Press, 2016.

[3] N. Forsgren, J. Humble and G. Kim, Accelerate: The Science of Lean Software and DevOps. Portland, OR: IT Revolution Press, 2018.

[4] NIST, "Secure Software Development Framework (SSDF), Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities," NIST SP 800-218, Feb. 2022. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-218.pdf

[5] L. Luettchau, "Hadolint — A smarter Dockerfile linter," accessed Nov. 4, 2026. [Online]. Available: https://github.com/hadolint/hadolint

[6] Bridgecrew, "Checkov — Infrastructure-as-Code static analysis," accessed Nov. 4, 2026. [Online]. Available: https://www.checkov.io

[7] Aqua Security, "Trivy — Comprehensive vulnerability scanner for containers and IaC," accessed Nov. 4, 2026. [Online]. Available: https://trivy.dev

[8] T. B. Brown et al., "Language Models are Few-Shot Learners," in Advances in Neural Information Processing Systems 33 (NeurIPS), 2020, pp. 1877–1901.

[9] GitHub, Inc., "GitHub Copilot — AI pair programmer," accessed Nov. 4, 2026. [Online]. Available: https://github.com/features/copilot

[10] Amazon Web Services, "Amazon CodeWhisperer," accessed Nov. 4, 2026. [Online]. Available: https://aws.amazon.com/codewhisperer

[11] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt and R. Karri, "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions," in

Proc. IEEE Symp. Security and Privacy (S&P), San Francisco, CA, 2022, pp. 754–768.

[12] J. Wilson, A. Chen and M. Patel, "Structured Output Constraints for Reliable LLM-Assisted Code Analysis," arXiv preprint arXiv:2403.11201, 2024.

[13] Center for Internet Security, "CIS Docker Benchmark v1.6.0," Nov. 2023. [Online]. Available: https://www.cisecurity.org/benchmark/docker

[14] Center for Internet Security, "CIS Kubernetes Benchmark v1.8.0," Jun. 2023. [Online]. Available: https://www.cisecurity.org/benchmark/kubernetes

[15] OWASP, "OWASP Top 10 CI/CD Security Risks," accessed Nov. 4, 2026. [Online]. Available: https://owasp.org/www-project-top-10-ci-cd-security-risks

[16] HashiCorp, "Terraform Security Best Practices," accessed Nov. 4, 2026. [Online]. Available: https://developer.hashicorp.com/terraform/cloud-docs/recommended-practices

[17] CNCF Technical Advisory Group for Security, "Cloud Native Security Whitepaper v2," Oct. 2022. [Online]. Available: https://github.com/cncf/tag-security

[18] S. Ramírez, "FastAPI documentation," accessed Nov. 4, 2026. [Online]. Available: https://fastapi.tiangolo.com

[19] Vercel, Inc., "Next.js 16 documentation," accessed Nov. 4, 2026. [Online]. Available: https://nextjs.org/docs

[20] D. Merkel, "Docker: Lightweight Linux containers for consistent development and deployment," Linux Journal, vol. 2014, no. 239, Art. 2, 2014.

[21] J. Humble and D. Farley, Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Boston, MA, USA: Addison-Wesley, 2010.

[22] GitHub, Inc., "GitHub Actions documentation," accessed Nov. 4, 2026. [Online]. Available: https://docs.github.com/actions

[23] Prometheus Authors, "Prometheus — Monitoring system and time series database," accessed Nov. 4, 2026. [Online]. Available: https://prometheus.io

[24] Grafana Labs, "Grafana — Open-source analytics and monitoring," accessed Nov. 4, 2026. [Online]. Available: https://grafana.com

[25] The OASIS Open Standard, "SARIF Version 2.1.0," Oct. 2019. [Online]. Available: https://docs.oasis-open.org/sarif/sarif/v2.1.0

APPENDIX A

BACKEND MODULE STRUCTURE

Table A.1 lists the main Python modules of the backend and what each contains.

Table A.1: Backend Python modules and their responsibilities

Module	Contents
app/main.py	FastAPI application factory, router registration, middleware, lifespan
app/models/analysis.py	Analysis, FindingRecord, FindingChatMessage SQLAlchemy models
app/models/configuration.py	Configuration SQLAlchemy model
app/models/user.py	User model with Argon2 password helpers
app/schemas/analysis.py	Pydantic v2 request/response schemas (AnalysisResponse, ChatMessage …)
app/api/routes/analyses.py	Analysis list, get, share, explain-score, SARIF, rule-versions endpoints
app/api/routes/findings.py	Finding chat GET and POST endpoints
app/api/routes/configurations.py	Configuration CRUD, analyze, trend endpoints
app/api/routes/auth.py	Register, login, me endpoints with JWT and Argon2
app/services/analyses.py	Analysis business logic: create, pipeline dispatch, trend query
app/services/chat.py	Remediation chat: ownership check, history, OpenAI call, persist
app/services/score_explanation.py	On-demand score explanation via OpenAI chat.completions
app/analysers/factory.py	RULE_VERSIONS dict, analyser factory function
app/analysers/dockerfile.py	9-rule Dockerfile analyser
app/analysers/docker_compose.py	7-rule Docker Compose analyser
app/analysers/github_actions.py	7-rule GitHub Actions analyser
app/analysers/kubernetes.py	9-rule Kubernetes analyser
app/analysers/terraform.py	7-rule Terraform / HCL analyser
app/analysers/helm.py	9-rule Helm chart analyser (regex, Go-template safe)
app/ai/provider.py	AIProvider ABC and AIAnalysisRequest / AIAnalysisResponse schemas
app/ai/openai_provider.py	OpenAI Responses API implementation
app/ai/ollama_provider.py	Ollama local-model implementation
app/ai/mock_provider.py	MockAIProvider for tests
migrations/versions/	Six Alembic migrations (initial schema → share_token)

APPENDIX B

REPOSITORY LAYOUT

Table B.1 lists the main folders and files of the project repository.

Table B.1: Repository layout

Path	Contents
frontend/	Next.js 16 App Router application (pages, components, lib)
frontend/app/	Route segments: analyses/[id], bulk, share/[token], new, dashboard …
frontend/components/	Shared components: Navbar, SeverityBadge
frontend/lib/	api.ts (typed fetch wrappers), types.ts (all TypeScript interfaces)
backend/	FastAPI application, analysers, AI providers, services, migrations
backend/app/	Main application package (models, schemas, api, services, ai, analysers)
backend/tests/	112 pytest tests
backend/migrations/	Alembic env.py and 6 migration version scripts
docker-compose.yml	Local dev stack: backend + PostgreSQL 16 + Prometheus + Grafana
Dockerfile	Multi-stage backend container image
.github/workflows/ci.yml	GitHub Actions: backend, frontend and docker jobs
provisioning/	Grafana datasource and dashboard auto-provisioning YAML files
.env.example	Environment variable template (DATABASE_URL, OPENAI_API_KEY …)

APPENDIX C

APPLICATION AND DEPLOYMENT SCREENSHOTS

The screenshots below show the deployed application, the CI pipeline and the observability dashboard. [Attach screenshots in this section before submission.]

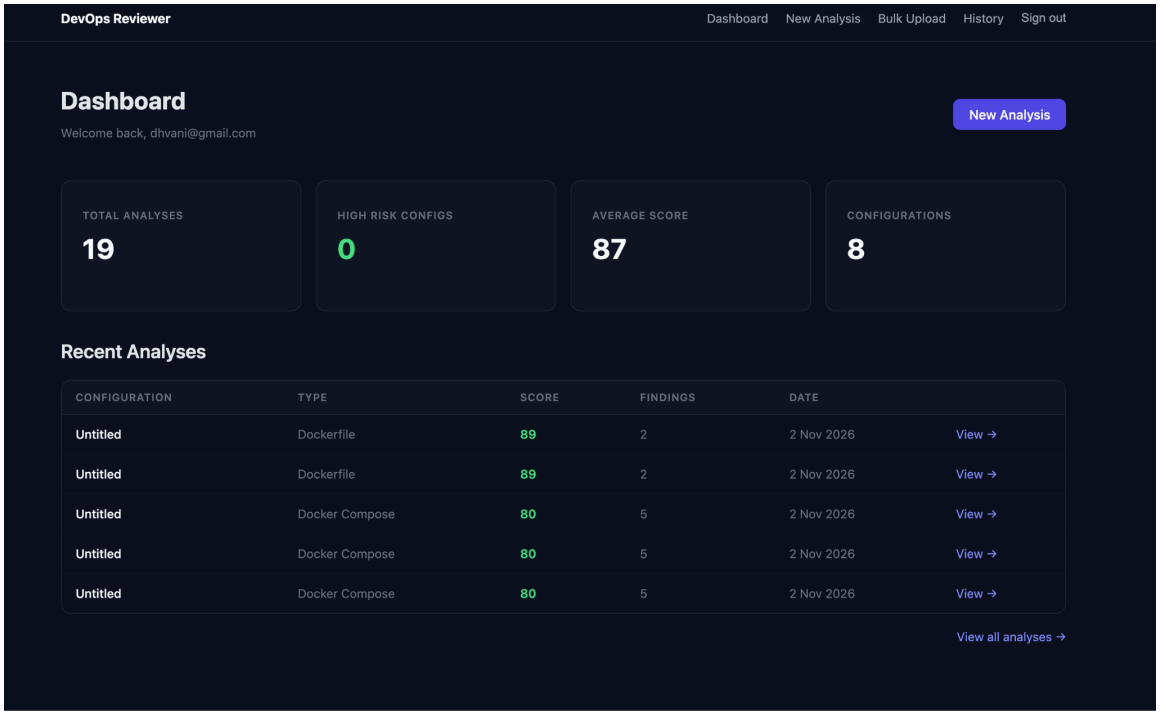


Fig. C.1 Analysis results page with finding cards, risk score and Explain button

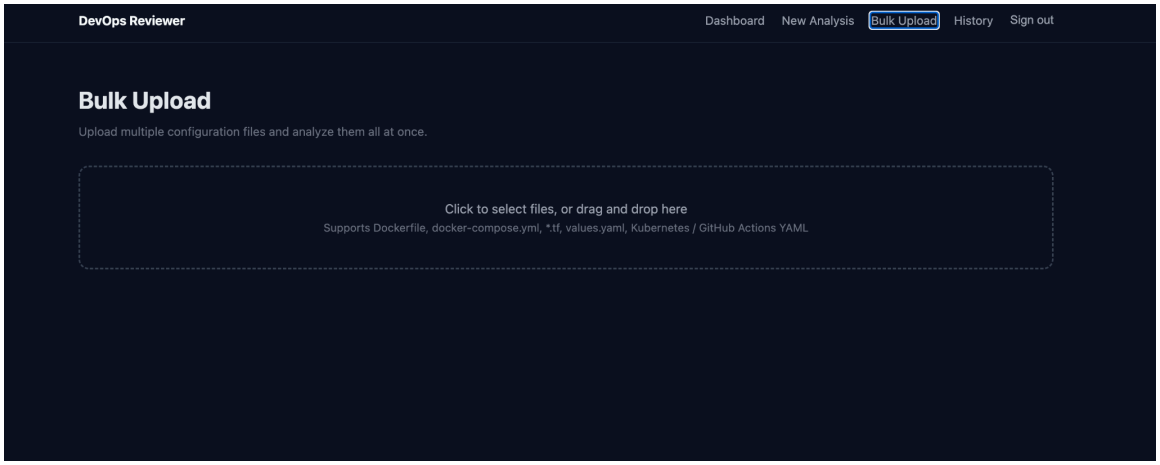


Fig. C.2 Bulk upload page showing parallel analysis progress and status badges

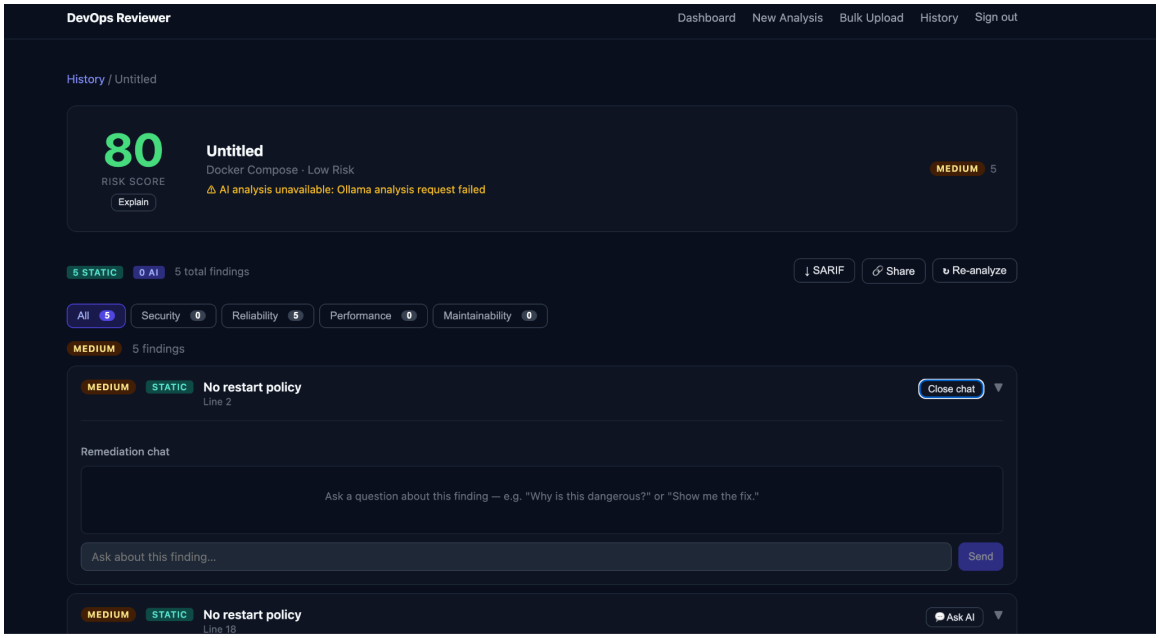


Fig. C.3 Per-finding remediation chat panel with multi-turn AI conversation

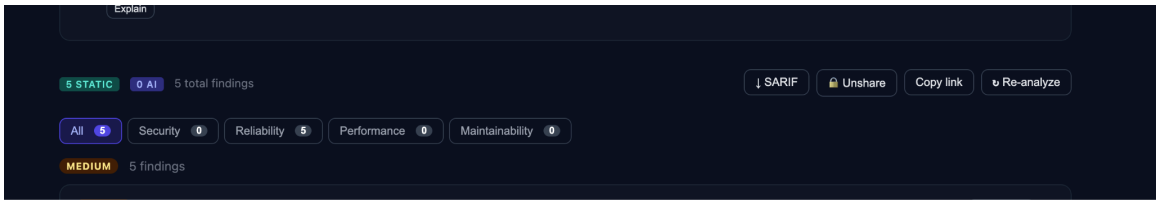


Fig. C.4 Public shareable report page accessible without login

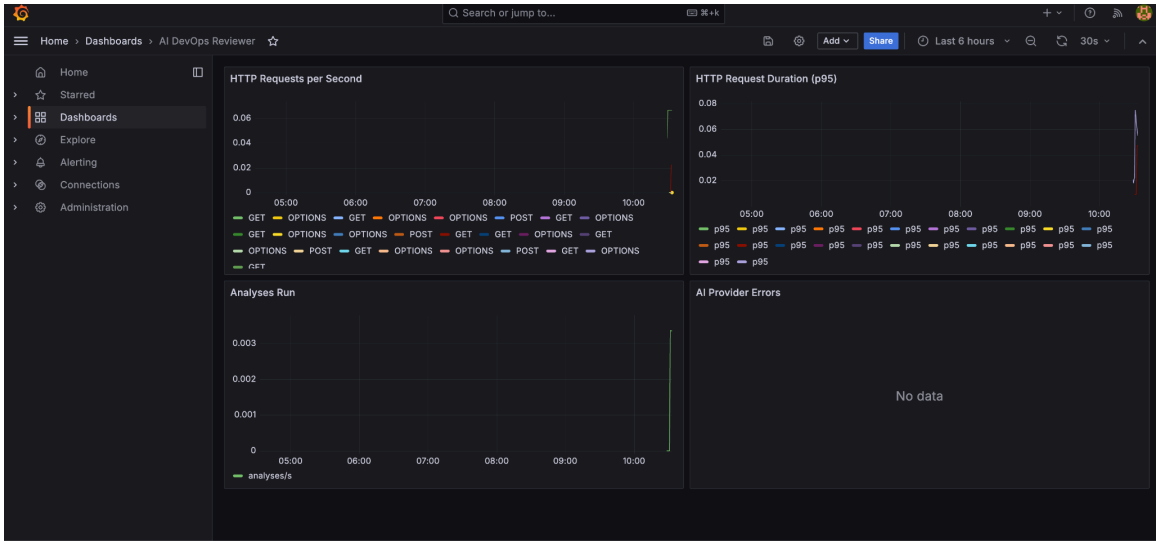


Fig. C.6 Grafana dashboard displaying Prometheus metrics for the backend

APPENDIX D

SELECTED SOURCE CODE

D.1 app/analysers/dockerfile.py — Dockerfile static analyser (excerpt)

```
from app.models.analysis import FindingRecord, FindingSeverity

RULES = [
    {
        "id": "DF001", "severity": "critical",
        "title": "Container runs as root",
        "pattern": r"^(?!.*USER\s+\S)",
        "description": "No USER instruction found. The container will run as
root.",
        "recommendation": "Add a non-root USER instruction, e.g. USER
1000:1000.",
    },
    {
        "id": "DF002", "severity": "high",
        "title": "Image pinned to 'latest' tag",
        "pattern": r"^FROM\s+\S+:latest",
        "description": "Using 'latest' makes the build non-reproducible.",
        "recommendation": "Pin to a specific digest or semantic version tag.",
    },
    # ... 7 more rules
]

class DockerfileAnalyser:
    def analyse(self, content: str) -> list[FindingRecord]:
        findings = []
        lines = content.splitlines()
        for rule in RULES:
            for i, line in enumerate(lines, 1):
                if re.search(rule["pattern"], line, re.IGNORECASE):
                    findings.append(FindingRecord(
                        rule_id=rule["id"],
                        severity=FindingSeverity(rule["severity"]),
                        title=rule["title"],
                        description=rule["description"],
                        recommendation=rule["recommendation"],
                        evidence=line.strip(),
                        line_number=i,
                        source="deterministic",
                    ))
        return findings
```

D.2 app/ai/openai_provider.py — OpenAI Responses API provider

```
from openai import AsyncOpenAI
from app.ai.provider import AIProvider, AIAnalysisRequest, AIAnalysisResponse

class OpenAIProvider(AIProvider):
    def __init__(self, settings):
```

```
        self._client = AsyncOpenAI(api_key=settings.openai_api_key)
        self._model  = settings.openai_model

    async def analyse(self, request: AIAnalysisRequest) -> AIAnalysisResponse:
        schema = AIAnalysisResponse.model_json_schema()
        response = await self._client.responses.create(
            model=self._model,
            input=[
                {"role": "system", "content": self._system_prompt(request)},
                {"role": "user",   "content": request.content},
            ],
            text={"format": {"type": "json_schema",
                             "name": "analysis",
                             "schema": schema,
                             "strict": True}},
        )
        return AIAnalysisResponse.model_validate_json(response.output_text)
```

D.3 app/services/chat.py — Per-finding remediation chat

```
from openai import AsyncOpenAI

async def send_chat_message(session, finding, user_message, settings):
    history = await get_chat_history(session, finding.id)
    messages = [{"role": "system", "content": _system_prompt(finding)}]
    for msg in history:
        messages.append({"role": msg.role, "content": msg.content})
    messages.append({"role": "user", "content": user_message})

    client = AsyncOpenAI(api_key=settings.openai_api_key)
    resp = await client.chat.completions.create(
        model=settings.openai_model,
        messages=messages,
        max_tokens=400,
    )
    assistant_text = resp.choices[0].message.content

    # Persist both turns
    user_row = FindingChatMessage(finding_id=finding.id,
                                  role="user", content=user_message)
    asst_row = FindingChatMessage(finding_id=finding.id,
                                  role="assistant", content=assistant_text)
    session.add_all([user_row, asst_row])
    await session.commit()
    await session.refresh(asst_row)
    return asst_row
```

D.4 .github/workflows/ci.yml — GitHub Actions CI pipeline

```
name: CI
on:
  push:
    branches: [main, dev]
  pull_request:
```

```
jobs:
  backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12", cache: pip }
      - run: pip install -r backend/requirements.txt
      - run: ruff check backend && ruff format --check backend
      - run: pytest backend/tests -q
        env: { DATABASE_URL: "sqlite+aiosqlite:///:memory:",
               AI_PROVIDER: mock,
               SECRET_KEY: testsecretkey32charslong12345678 }

  frontend:
    runs-on: ubuntu-latest
    defaults: { run: { working-directory: frontend } }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: npm,
                cache-dependency-path: frontend/package-lock.json }
      - run: npm ci
      - run: npm run lint
      - run: npx tsc --noEmit
      - run: npm run build

  docker:
    runs-on: ubuntu-latest
    needs: [backend]
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t devops-reviewer-backend .
```